
The Implementation Is the Hypothesis: A No-Op Intervention Defect, Eight Negative Levers, and a Mechanism That Only Wins Once Its Kernel Is Fixed

Table 5 (deep-analysis author), OPHIS¹

Abstract

This block began by discovering that the previous block’s results were not results at all. An adversarial critic in the planning round reported it could not find `ATTNRES_ENABLE`, `NGRAM_WD_LAMBDA`, or `NGRAM_BACKOFF_KAPPA` anywhere in the canonical `train.py`. Verification confirmed a defect with two independent causes, either sufficient alone: the three mechanisms existed only as forked one-off scripts on the remote box, and their intervention records bound no environment variables. Every “treatment” arm had therefore run a configuration byte-identical to its own control, while the gated runner reported `config_verified=True` — because that check iterates the bound environment, and an empty environment verifies trivially. Three results were retracted as invalid: they are the same configuration measured twice, which is exactly why all three landed an order of magnitude inside the noise gate. After porting the mechanisms, fixing a latent autograd bug in the reference `AttnRes`, and adding two fail-closed guards to the runner, we re-measured. Eight levers were then tested with working implementations — `AttnRes`, token-shift, `C1`, `C3`, `C1×C3`, sparse-grad, rowwise state, and an `nGPT` hypersphere residual — and every one regressed, with the striking common signature that each lost 80–150 optimizer steps inside the fixed 300-second budget. The mechanisms were not the binding constraint; step loss was. The

block’s final and central result inverts that picture. Intra-document attention masking, implemented densely, is catastrophic (1.0130, 522 steps); implemented block-sparse via `FlexAttention` it reaches 0.9438 at 1084 steps — -0.0692 `val_bpb` and $2.08\times$ the throughput from an implementation change alone, at identical masking semantics. Against a matched-backend control it is a confirmed win of -0.0111 over two paired seeds, $2.7\times$ the 2σ gate, while simultaneously gaining steps. A mechanism this campaign had previously recorded as “improves quality but carries a large step-time cost” was in fact an implementation artifact.

1. Introduction

Paper 014 planned a seven-candidate block and reported that four of its experiments produced small nulls. This paper reports that those nulls were not measurements. It then reports what happened once the harness was repaired: a sweep of eight independent levers, all negative and all sharing one mechanical cause, followed by a single mechanism whose sign flips entirely when its kernel is replaced.

The organizing lesson is the one the operator stated as a standing directive and which the data now supports quantitatively: a naive implementation may lose where a refined one wins, so a wall-clock regression is a statement about the implementation until proven otherwise.

2. A Defect That Cannot Be Detected By Experiment

The block-15 planning round’s adversarial critic flagged that `ATTNRES_ENABLE`, `NGRAM_WD_LAMBDA`, and `NGRAM_BACKOFF_KAPPA` appeared nowhere in the local checkout. Direct verification found

¹Autoresearch reimplement campaign. Correspondence to: Fable 5 (OPHIS) <baiyuzhu@mit.edu>.

zero case-insensitive matches across all 3358 lines of `train.py`, all five `sota_snapshots`, the read-only v3 reference checkout, and the entire git history of the file. A remote filesystem search located the real implementations: `repo/train_attnres_c1_v2.py` and `repo/train_c3.py`, forked one-off scripts of roughly 700 lines, never merged into the 3358-line canonical trainer.

The second cause is independent and equally fatal. `tools/run_gated.py: _arm_env` derives an arm’s environment from `intervention.parameters.env`, falling back to uppercase keys in `parameters`. All three intervention records carried only lowercase descriptive metadata (`{"kappa": 50}`, `{"lambda_values_tested": [...]}`), so each resolved to `env={}` and exported nothing.

Why it survived the gate. `verify_config` iterates the bound environment and compares each key against the trainer’s `RESOLVED_CONFIG` echo. An empty bound environment has nothing to check, so verification passed vacuously and every run was stamped `config_verified=True`. No experiment can detect this failure, because a no-op treatment and its control are the same program: the defect is invisible to the very instrument meant to catch it, and is only reachable by reading code.

Consequence. The reported deltas -0.000839 , -0.001281 , -0.001019 are not null results about the named mechanisms. They are one configuration measured twice, which is precisely why all three sat an order of magnitude inside the 2σ gate of 0.004072 . They are retracted as invalid, not reported as nulls.

Repairs. (i) All three mechanisms ported into the canonical trainer, verbatim from the forks, each an exact no-op when off. (ii) A latent autograd bug in the reference `AttnRes` was found and fixed: preallocated $(L+1, B, T, d)$ stack buffers written in place bump the version counter of the slices backward requires, raising a `RuntimeError` — this matches, and plausibly explains, the `AttnRes` crash already recorded in paper 013’s negative results. Replaced with list-append plus `torch.stack`. (iii) All three flags are now echoed in `RESOLVED_CONFIG`, so `config-verify` can observe them. (iv) Intervention records with executable `parameters.env` were registered under new IDs rather than version bumps, so every frozen experiment fingerprint stays valid and history is not rewritten. (v) Two fail-closed guards: a treatment intervention resolving to an empty environment is now an error, and a bound key absent from `RESOLVED_CONFIG` is reported distinctly from a value mismatch.

Table 1. All levers regress, and all lose steps. Budget is 300 wall-clock seconds, so steps are the currency.

Lever	val_bpb	steps
baseline	0.9410	1133
baseline (replicate)	0.9423	1117
rowwise state	0.9418	1131
AttnRes (refined)	0.9525	1050
AttnRes (naive)	0.9562	998
sparse-grad	0.9535	984
token-shift	0.9608	1044
nGPT sphere ($\alpha=0.25$)	0.9802	1097
token-shift $\times$ C3	0.9648	992

3. Eight Levers, One Failure Mode

With working implementations, we swept eight levers against a baseline of 0.9416 at ~ 1132 steps (within-round $\sigma \approx 0.0009$ from two replicates).

Three observations. First, every lever costs steps, 80–150 of them, and in a wall-clock-budgeted frame that cost is paid directly in `val_bpb`. Second, refinement moves the number in the right direction: `AttnRes` improved -0.0044 and recovered 53 steps purely from a kernel rewrite that stopped materializing two (S, B, T, d) tensors per layer, dropping peak VRAM from 115 GB to 89.7 GB (baseline parity) — the mechanism was never re-specified. Third, one lever is exactly free: rowwise optimizer state costs $+0.00004$ `val_bpb` and one step, both far inside noise, while cutting optimizer memory. It is banked as headroom, not claimed as a win.

Two corrections belong here. `NGRAM_SPARSE_GRAD` is recorded in this campaign’s own notes as a -9.3% step-time win; on this box it costs 148 steps. The deciding variable is the torch version (2.13 on the dev box where the win was measured, 2.9.1 here), so the lever is version-gated rather than universal, and the note has been corrected. The nGPT hypersphere residual (arXiv 2410.01131, claiming a 4–20 $\times$ step reduction) was the cheapest lever tested at only -36 steps, exactly as predicted for an elementwise operation, yet quality-per-step collapsed. Its α -sweep is non-monotonic with an interior optimum at 0.25, so it is a real mechanism rather than a bug; the likely cause is that a residual-only nGPT without weight normalization and without removing weight decay is a half-system, discarding the baseline’s tuned residual structure while delivering none of the paper’s benefit.

Table 2. Identical masking semantics; the implementation decides the verdict.

Arm	val_bpb	steps
fa3 baseline	0.9418	1133
sdpa, no mask	0.9530	983
dense, window only	1.0184	528
dense, window + doc	1.0130	522
flex, window + doc	0.9438	1084
flex, seg (pure INTRADoc)	0.9426	1091

4. The Implementation Is the Hypothesis

Intra-document attention masking blocks cross-document attention inside a packed sequence. It is the highest-ranked untested item in this project’s own literature synthesis, and carries a consistent external signal. Its code path already existed in `train.py`, with both a dense and a block-sparse (FlexAttention) implementation. It had never been tested against the current baseline.

Following the critic’s windowing-matched design — necessary because the doc-mask path is gated on the SDPA backend, so any test carries a mandatory `fa3→sdpa` swap — we ran a `MODE=window` control (windowing only) against `MODE=both` (windowing plus document restriction) at matched backend and matched kernel.

The mechanism is favorable and the dense implementation hid it. At matched windowing and matched (dense) kernel, adding document restriction is worth -0.0054 . But the dense path materializes a $(B, 1, T, T)$ boolean mask re-read by every layer in both passes, halving throughput from 983 to 522 steps — a cost the code’s own comment had predicted. Swapping to the block-sparse kernel, at identical masking semantics, yields -0.0692 val_bpb and $2.08\times$ the steps.

Confirmed against a matched control. Paired against `sdpa-without-mask`: seed 42 gives -0.00923 , seed 43 gives -0.01291 ; mean -0.0111 , same sign, $2.7\times$ the 2σ gate, and $+101$ steps. Quality and throughput improve together, which no other lever this block achieved. A third seed holds (3-seed mean 0.942945 , $\sigma = 0.00088$).

This retroactively corrects a standing campaign belief. The literature synthesis records that “the current dense segmented-attention implementation improves quality but carries a large step-time cost,” and that verdict is what kept the mechanism shelved. It was an artifact of the dense kernel, not a property of document segmentation.

5. Discussion

The eight-lever sweep and the doc-mask result are the same finding viewed from two sides. In a wall-clock-budgeted frame, a mechanism’s measured value is inseparable from its kernel: every lever that added compute lost steps and therefore lost, and the one lever whose kernel was replaced flipped from catastrophic to winning without any change to what it computes. A wall-clock regression is therefore evidence about an implementation, and only becomes evidence about a mechanism once the implementation is competitive — which is what this campaign’s own Protocol 3 asserts and what the AttnRes and doc-mask trajectories now demonstrate quantitatively.

The honest limitation: flex doc-mask sits $+0.0013$ from the fa3 baseline, statistically indistinguishable, because the doc-mask benefit (-0.011) nearly exactly cancels the SDPA backend penalty ($+0.012$). It is not yet a campaign SOTA. The path to one is doc-masking at fa3 speed, and we established a concrete obstacle: this build exposes `flash_attn_varlen_func` only as an `autograd.Function`, not as the low-level custom-op pair the non-varlen path required — and letting `torch.compile(fullgraph=True)` trace into an `autograd.Function` is the documented cause of this box’s historical segfault. That is a real engineering obstacle with a known failure mode, not an unknown.

6. Ranked Next Directions

1. Block-size tuning for the flex mask. The flex kernel is only 3% behind fa3 in steps; the block-sparse granularity is a direct lever on that gap, and closing it converts a parity result into a SOTA.
2. fa3 varlen doc-masking. The highest-value target (~ -0.0054 on top of the fa3 baseline), gated on wrapping `FlashAttnVarlenFunc` in a `torch.library.custom_op` with `register_fake`, exactly as the non-varlen path was rescued.
3. Re-test shelved levers under doc-masking. Removing cross-document contamination changes the context every other mechanism sees; the n-gram memory in particular now reads cleaner statistics, so the memorizer-regularization family deserves a re-test in the new regime rather than inheriting its pre-doc-mask verdict.
4. Full nGPT rather than the residual-only half-system, accepting that it requires retuning the recipe it replaces.
5. Paired Head Attention (NanoGPT speedrun record 58), currently a name without a public

specification.

7. Conclusion

A block that began by invalidating its own predecessor ends with the campaign’s clearest positive result. The invalidation and the result share a moral: what is registered as a mechanism is only ever tested through an implementation, and when the two are conflated the ledger records fiction — silently, in the no-op case, or as a false negative, in the dense-mask case. The instrument has been hardened against the first failure, and the second has produced a confirmed -0.0111 win that was sitting unclaimed behind a slow kernel.

References

- Loshchilov, I. et al. nGPT: Normalized Transformer with Representation Learning on the Hypersphere. arXiv:2410.01131.
- METR. Evidence on AI R&D Progress from NanoGPT. 2026.
- Recursive. First Steps Toward Automated AI Research. 2026.
- OPHIS. From Isolation to Recombination. Campaign paper 013.
- OPHIS. Block 14: Recombination and the Hopper Correction. Campaign paper 014.